



**ADAMA SCIENCE AND TECHNOLOGY UNIVERSITY
SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(SOFTWARE ENGINEERING)**

**COURSE_TITLE: Advanced Programming
COURSE_CODE: CSEg3203
SECTION: 1
Group: 2**

**Submitted by : Anom Abebe
ID: UGR/33940/16**

**Submitted to: Yared Daba Tesfaye
Submission date: 2025-12-19**

Java Client–Server Weather Information System

1. Introduction

The Weather Information System is a distributed Java application designed to simulate real-time weather data retrieval. It utilizes a Client–Server architecture based on TCP (Transmission Control Protocol) sockets. The system allows users to query weather conditions for various cities via a graphical user interface (GUI) built with Java Swing, while a multi-threaded server handles the data processing and retrieval.

2. System Architecture

The application is divided into two distinct components: the Server and the Client.

2.1 The Weather Server (WeatherServer.java)

- **Role:** Acts as the backend data provider.
- **Networking:** Listens on Port 5000 using a ServerSocket.
- **Concurrency:** Implements a Multi-threaded architecture. Every time a client connects, the server spawns a new Thread using the Runnable interface. This ensures that multiple clients can request data simultaneously without blocking one another.
- **Data Storage:** Uses a local, in-memory HashMap<String, String> as a mock database. No external APIs are contacted, ensuring offline functionality and speed.

2.2 The Weather Client (WeatherClient.java)

- **Role:** Acts as the frontend user interface.
 - **UI Framework:** Built using Java Swing (JFrame, JTextArea, JTextField).
 - **Networking:** Establishes a Socket connection to localhost on Port 5000.
 - **Communication:** Runs network I/O operations on a separate thread (using SwingUtilities for UI updates) to prevent the interface from freezing while waiting for server responses.
1. **User Input:** Provides a text field for entering city names.
 2. **Display Area:** Features a scrollable text area to show the conversation history with the server.
 3. **Connectivity Handling:** Gracefully handles connection errors if the server is offline.
 4. **Input Validation:** Prevents sending empty requests.

3. Mock Data Sample

The server simulates data internally. An example of the data structure includes:

- **Key:** addis ababa
- **Value:**
code Text

City: Addis Ababa
Temperature: 24°C
Humidity: 45%
Condition: Sunny
Wind Speed: 12 km/h
-